

Práctica de Programación Orientada a Objetos

Sistema de Gestión de Eventos

En esta práctica se desarrollará de manera progresiva un sistema para gestionar eventos y sus reservas.

Los ejercicios están relacionados entre sí. Cada ejercicio incorpora nuevas funcionalidades al programa desarrollado en los ejercicios anteriores. Por lo tanto, las clases implementadas deberán conservarse y modificarse cuando sea necesario.

Ejercicio 1 – Modelar un evento

Implemente una clase `Evento` que permita representar un evento.

Cada evento debe almacenar la siguiente información:

- `codigo_evento (str)`: código de identificación único del evento.
- `nombre (str)`: nombre del evento.
- `fecha (str)`: fecha del evento en formato "YYYY-MM-DD".
- `hora (str)`: hora de inicio del evento en formato "HH:MM".
- `capacidad (int)`: cantidad máxima de asistentes.

La clase debe incluir los métodos:

- `__init__(...)`
- `__str__()`

El método `__str__` debe devolver una representación en cadena que permita visualizar claramente la información del evento.

Por ejemplo:

```
1 evento1 = Evento(  
2     "E001",  
3     "Concierto de Rock",  
4     "2026-10-20",  
5     "19:00",  
6     100  
7 )  
8  
9 print(evento1)
```

Ejercicio 2 – Modelar una reserva

A partir de la clase `Evento` implementada en el ejercicio anterior, implemente una clase `ReservaEvento`.

Cada reserva debe almacenar:

- `dni_cliente (str)`: DNI de la persona que realiza la reserva.
- `nombre_cliente (str)`: nombre de la persona que realiza la reserva.
- `evento (Evento)`: evento para el cual se realiza la reserva.
- `numero_asistentes (int)`: cantidad de personas incluidas en la reserva.

Observe que el atributo `evento` debe almacenar un **objeto de tipo `Evento`**, y no solamente el código del evento.

La clase debe implementar:

- `__init__(...)`
- `__str__()`

El método `__str__` debe mostrar los datos de la reserva e incluir información correspondiente al evento reservado.

Por ejemplo:

```

1 reserva1 = ReservaEvento(
2     "12345678",
3     "Juan Perez",
4     evento1,
5     4
6 )
7
8 print(reserva1)

```

Ejercicio 3 – Crear un sistema que administre eventos

Implemente una clase `SistemaEventos` que permita administrar múltiples eventos.

En esta primera versión, el sistema solamente almacenará los eventos disponibles.

La clase debe tener un atributo:

- **eventos**: diccionario que contiene los eventos registrados en el sistema.

La **clave** de cada elemento del diccionario será el código del evento y su **valor** será el objeto de tipo `Evento` correspondiente.

Por ejemplo, conceptualmente:

```

1 {
2     "E001": evento1,
3     "E002": evento2
4 }

```

Implementar los siguientes métodos:

- `__init__()`: crea un sistema sin eventos.
- `agregar_evento(evento)`: recibe un objeto de tipo `Evento` y lo incorpora al sistema.
- `mostrar_eventos()`: muestra por pantalla todos los eventos registrados en el sistema.

Crear al menos tres eventos diferentes, agregarlos al sistema y utilizar `mostrar_eventos()` para comprobar su funcionamiento.

Ejercicio 4 – Eliminar eventos

Agregar a la clase `SistemaEventos` el método:

`eliminar_evento(codigo_evento)`

El método recibe el código de un evento y debe eliminar del sistema el evento correspondiente. Si no existe ningún evento con ese código, el método no debe realizar ninguna acción.

El método no devuelve ningún valor.

Realizar pruebas intentando eliminar:

- un evento existente;
- un evento inexistente.

Luego mostrar nuevamente los eventos del sistema.

Ejercicio 5 – Incorporar las reservas al sistema

Modifique la clase `SistemaEventos` para que, además de administrar eventos, permita almacenar las reservas realizadas para cada uno de ellos.

Agregar a la clase un atributo:

- **reservas**: diccionario que almacena las reservas correspondientes a cada evento.

La clave será el código del evento y el valor asociado será una **lista de objetos de tipo `ReservaEvento`**.

Por ejemplo, conceptualmente el diccionario podría tener la siguiente estructura:

```
1 {  
2     "E001": [reserva1, reserva2],  
3     "E002": [reserva3]  
4 }
```

Modifique el método `__init__()` de manera que un nuevo sistema se cree inicialmente sin eventos y sin reservas.

Ejercicio 6 – Calcular la capacidad disponible

Agregar a la clase `SistemaEventos` el método:

`devolver_capacidad_restante(codigo_evento)`

El método recibe el código de un evento y debe devolver la cantidad de lugares que todavía se encuentran disponibles para realizar reservas.

Para determinar la capacidad restante deben considerarse la capacidad máxima del evento y todas las reservas que ya fueron realizadas para dicho evento.

Por ejemplo, si un evento tiene una capacidad máxima de 100 personas y se realizaron reservas para:

- 4 asistentes,
- 3 asistentes,
- 5 asistentes,

la capacidad restante del evento será:

$$100 - (4 + 3 + 5) = 88$$

Por lo tanto, el método deberá devolver 88.

Ejercicio 7 – Crear una reserva

Agregar a la clase `SistemaEventos` el método:

`crear_reserva(dni, nombre, codigo_evento, cantidad)`

El método permite realizar una nueva reserva para un determinado evento.

Antes de crear la reserva se debe verificar que:

1. exista un evento con el código indicado;
2. exista capacidad suficiente para la cantidad de asistentes solicitada.

Solamente si ambas condiciones se cumplen se debe crear el objeto de tipo **ReservaEvento** correspondiente e incorporarlo al sistema.

Si alguna de las condiciones anteriores no se cumple, la reserva no debe realizarse.

El método no devuelve ningún valor.

Pruebe el funcionamiento del método considerando, al menos, las siguientes situaciones:

```
1  # Reserva valida
2  sistema.crear_reserva(
3      "12345678",
4      "Juan Perez",
5      "E001",
6      4
7  )
8
9  # Evento inexistente
10 sistema.crear_reserva(
11     "11111111",
12     "Ana Garcia",
13     "E999",
14     2
15 )
16
17 # Cantidad superior a la capacidad disponible
18 sistema.crear_reserva(
19     "22222222",
20     "Pedro Gomez",
21     "E001",
22     200
23 )
```

Ejercicio 8 – Eliminar una reserva

Agregar a la clase **SistemaEventos** el método:

```
eliminar_reserva(dni_cliente, codigo_evento)
```

El método debe eliminar la reserva correspondiente al cliente cuyo DNI se recibe como parámetro para el evento indicado.

Si el evento no existe o no existe una reserva correspondiente a ese cliente para dicho evento, el método no debe realizar ninguna acción.

El método no devuelve ningún valor.

Luego de eliminar una reserva, utilice el método **devolver_capacidad_restante** para comprobar que la capacidad disponible del evento se haya actualizado correctamente.

Ejercicio 9 – Integración del sistema

Una vez implementadas todas las funcionalidades anteriores, utilice las clases desarrolladas para ejecutar el siguiente programa:

```
1  sistema = SistemaEventos()
2
3  # Crear eventos
4  evento1 = Evento(
5      "E001",
6      "Concierto de Rock",
7      "2026-10-20",
```

```

8      "19:00",
9      100
10 )
11
12 evento2 = Evento(
13     "E002",
14     "Teatro Musical",
15     "2026-10-21",
16     "20:00",
17     50
18 )
19
20 # Agregar los eventos al sistema
21 sistema.agregar_evento(evento1)
22 sistema.agregar_evento(evento2)
23
24 # Mostrar los eventos disponibles
25 sistema.mostrar_eventos()
26
27 # Crear reservas
28 sistema.crear_reserva(
29     "12345678",
30     "Juan Perez",
31     "E001",
32     4
33 )
34
35 sistema.crear_reserva(
36     "87654321",
37     "Maria Lopez",
38     "E002",
39     3
40 )
41
42 # Consultar capacidad restante
43 print(
44     "Capacidad restante para E001:",
45     sistema.devolver_capacidad_restante("E001")
46 )
47
48 # Eliminar una reserva
49 sistema.eliminar_reserva(
50     "12345678",
51     "E001"
52 )
53
54 # Consultar nuevamente la capacidad
55 print(
56     "Capacidad restante para E001:",
57     sistema.devolver_capacidad_restante("E001")
58 )

```

Analice el resultado obtenido y compruebe que las reservas realizadas y eliminadas produzcan los cambios esperados en la capacidad disponible de cada evento.

Sistema de Gestión de Biblioteca

En los siguientes ejercicios se desarrollará progresivamente un sistema para administrar los libros y préstamos de una biblioteca.

Los ejercicios están relacionados entre sí. Cada ejercicio incorpora nuevas funcionalidades a las clases desarrolladas anteriormente.

Ejercicio 10 – Modelar un libro

Implemente una clase `Libro` que permita representar un libro.
Cada libro debe almacenar:

- `codigo_libro (str)`: código único del libro.
- `titulo (str)`: título del libro.
- `autor (str)`: autor del libro.

La clase debe implementar los métodos:

- `__init__(...)`
- `__str__()`

El método `__str__` debe devolver una representación en cadena que permita visualizar claramente la información del libro.

Por ejemplo:

```
1 novela1 = Libro(  
2     "N001",  
3     "Cien años de soledad",  
4     "Gabriel García Márquez"  
5 )  
6  
7 print(novela1)
```

Ejercicio 11 – Representar los ejemplares de un libro

Una biblioteca puede disponer de varios ejemplares de un mismo libro.

Implemente una clase `Ejemplares` que permita administrar la cantidad de ejemplares existentes de un determinado libro.

La clase debe almacenar:

- `libro (Libro)`: objeto que representa el libro.
- `total (int)`: cantidad total de ejemplares que posee la biblioteca.
- `prestados (int)`: cantidad de ejemplares que se encuentran actualmente prestados.

Al crear un objeto de tipo `Ejemplares`, inicialmente ningún ejemplar se encuentra prestado.

Por ejemplo:

```
1 ejemplar_novela1 = Ejemplares(novela1, 5)
```

En este caso, la biblioteca posee cinco ejemplares del libro `novela1` y ninguno de ellos se encuentra inicialmente prestado.

Ejercicio 12 – Consultar ejemplares disponibles

Agregar a la clase `Ejemplares` el método:

`disponibles()`

El método debe devolver la cantidad de ejemplares que actualmente se encuentran disponibles. Por ejemplo, si existen 5 ejemplares de un libro y 2 de ellos se encuentran prestados:

```
1 print(ejemplar_novela1.disponibles())
```

debe mostrar:

```
1 3
```

Ejercicio 13 – Prestar y devolver ejemplares

Agregar a la clase `Ejemplares` los siguientes métodos:

- `prestar()`: registra el préstamo de un ejemplar del libro.
- `devolver()`: registra la devolución de un ejemplar previamente prestado.

El método `prestar()` solamente debe modificar la cantidad de ejemplares prestados cuando exista al menos un ejemplar disponible.

El método `devolver()` solamente debe modificar la cantidad de ejemplares prestados cuando exista al menos un ejemplar actualmente prestado.

Ninguno de los métodos devuelve un valor.

Por ejemplo:

```
1 ejemplar_novela1.prestar()
2 ejemplar_novela1.prestar()
3
4 print(ejemplar_novela1.disponibles())
5
6 ejemplar_novela1.devolver()
7
8 print(ejemplar_novela1.disponibles())
```

Analice cuál debería ser la salida producida en cada caso.

Ejercicio 14 – Incorporar nuevos ejemplares

Agregar a la clase `Ejemplares` el método:

`agregar_ejemplares(cantidad)`

El método recibe una cantidad y debe incrementar en dicha cantidad el número total de ejemplares existentes del libro.

Por ejemplo:

```
1 ejemplar_novela1.agregar_ejemplares(3)
```

Si originalmente existían 5 ejemplares, luego de ejecutar la instrucción anterior deben existir 8 ejemplares en total.

Ejercicio 15 – Crear una biblioteca

Implemente una clase `Biblioteca` que permita administrar los ejemplares existentes. En esta primera versión, la clase debe contener un atributo:

- `ejemplares`: diccionario que almacena los ejemplares registrados en la biblioteca.

La clave de cada elemento será el código de un libro y el valor será el objeto de tipo `Ejemplares` correspondiente.

Conceptualmente, el diccionario podría tener la siguiente estructura:

```
1 {  
2     "N001": ejemplar_novela1,  
3     "T001": ejemplar_texto1  
4 }
```

Implementar:

- `__init__()`: crea una biblioteca inicialmente vacía.
- `agregar_ejemplares(ejemplares)`: recibe un objeto de tipo `Ejemplares` y lo incorpora al diccionario.

Por ejemplo:

```
1 biblioteca = Biblioteca()  
2  
3 biblioteca.agregar_ejemplares(ejemplar_novela1)  
4 biblioteca.agregar_ejemplares(ejemplar_texto1)
```

Ejercicio 16 – Registrar los préstamos

Modifique la clase `Biblioteca` para que también permita registrar qué libros tiene prestados cada persona.

Agregar el atributo:

- `prestamos`: diccionario cuya clave es el DNI de una persona y cuyo valor es una lista con los códigos de los libros que dicha persona tiene actualmente prestados.

Por ejemplo, conceptualmente:

```
1 {  
2     "12345678": ["N001", "T001"],  
3     "87654321": ["N001"]  
4 }
```

En este ejemplo, la persona con DNI "12345678" tiene dos libros prestados, mientras que la persona con DNI "87654321" tiene uno.

Modifique el método `__init__()` para que una nueva biblioteca se cree inicialmente sin ejemplares y sin préstamos.

Ejercicio 17 – Realizar el préstamo de un libro

Agregar a la clase `Biblioteca` el método:

`prestamo_libro(dni, codigo_libro)`

El método permite prestar un ejemplar del libro indicado a la persona identificada por su DNI. Antes de realizar el préstamo se debe verificar que:

1. el código corresponda a un libro registrado en la biblioteca;

2. exista al menos un ejemplar disponible de dicho libro.

Solamente si ambas condiciones se cumplen debe registrarse el préstamo.

Al realizar un préstamo correctamente:

- debe actualizarse la cantidad de ejemplares prestados del libro;
- debe registrarse el código del libro en la lista de préstamos correspondiente al DNI indicado.

Tenga en cuenta que puede tratarse del **primer préstamo realizado por esa persona**.

Si el libro no existe o no quedan ejemplares disponibles, el préstamo no debe realizarse.

El método no devuelve ningún valor.

Ejercicio 18 – Devolver un libro

Agregar a la clase Biblioteca el método:

```
devolver_libro(dni, codigo_libro)
```

El método permite registrar la devolución de un libro.

La devolución solamente debe realizarse si el préstamo indicado se encuentra efectivamente registrado para la persona correspondiente.

Cuando la devolución sea válida:

- debe actualizarse la cantidad de ejemplares prestados del libro;
- debe eliminarse el código del libro de la lista de préstamos correspondiente a esa persona.

Si no existe el préstamo indicado, no debe realizarse ninguna modificación.

El método no devuelve ningún valor.

Ejercicio 19 – Integración del sistema

Una vez implementadas todas las funcionalidades anteriores, ejecute el siguiente programa:

```
1  # Crear libros
2  novela1 = Libro(
3      "N001",
4      "Cien años de soledad",
5      "Gabriel García Márquez"
6  )
7
8  texto1 = Libro(
9      "T001",
10     "Matemáticas para Todos",
11     "José Pérez"
12 )
13
14 # Crear los ejemplares
15 ejemplar_novela1 = Ejemplares(novela1, 5)
16 ejemplar_texto1 = Ejemplares(texto1, 3)
17
18 # Crear la biblioteca
19 biblioteca = Biblioteca()
20
21 # Incorporar ejemplares
22 biblioteca.agregar_ejemplares(ejemplar_novela1)
23 biblioteca.agregar_ejemplares(ejemplar_texto1)
24
25 # Realizar préstamos
26 biblioteca.prestamo_libro(
```

```

27     "12345678",
28     "N001"
29 )
30
31 biblioteca.prestamo_libro(
32     "12345678",
33     "T001"
34 )
35
36 # Consultar disponibilidad
37 print(ejemplar_novela1.disponibles())
38 print(ejemplar_texto1.disponibles())
39
40 # Devolver un libro
41 biblioteca.devolver_libro(
42     "12345678",
43     "N001"
44 )
45
46 # Consultar nuevamente la disponibilidad
47 print(ejemplar_novela1.disponibles())
48 print(ejemplar_texto1.disponibles())

```

Determine qué valores deberían mostrarse en cada una de las consultas realizadas y verifique que coincidan con el funcionamiento del programa.